

Proyecto 1

Uso de un protocolo existente

Chatbot anfitrión con MCP/JSON-RPC manual
y servidor industrial Plant Energy MCP

Estudiante	Diego Alejandro Ramírez Velásquez
Carné	23601
Fecha	27 de agosto de 2026
Repositorio	https://github.com/Alejoram2505/redes

Resumen

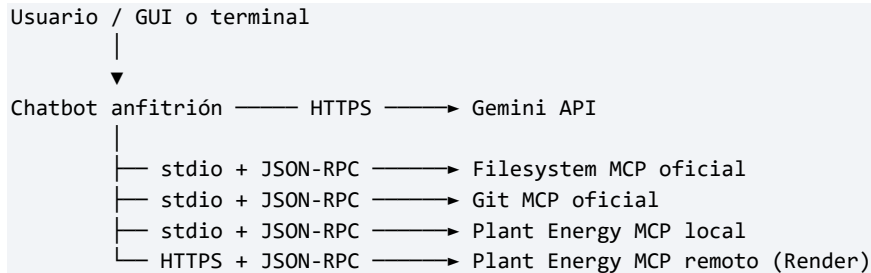
El proyecto implementa un chatbot anfitrión conectado a Gemini mediante API. El anfitrión mantiene contexto durante la sesión, descubre herramientas y ejecuta solicitudes sobre tres familias de servidores MCP: Filesystem oficial, Git oficial y Plant Energy MCP, este último disponible tanto como proceso local por stdio como servicio remoto por HTTPS en Render. La implementación del protocolo MCP y del intercambio JSON-RPC es manual; no se utilizó FastMCP ni un SDK que oculte el protocolo.

Matriz de cumplimiento de los puntos 1 a 10

Punto	Requisito	Estado	Evidencia
1	Conexión real con LLM	Implementado y probado	Gemini respondió preguntas generales.
2	Contexto por sesión	Implementado y probado	Continuación sobre Alan Turing conservó contexto.
3	Log MCP visible	Implementado y probado	Disponible desde /log y desde la GUI.
4	Filesystem y Git oficiales	Implementado y probado	Demo aislada creó README, stage y commit.
5	Servidor MCP local propio	Implementado y probado	Plant Energy MCP expone cinco herramientas.
6	Mismo servidor remoto	Implementado y probado	Render /health=200 y paridad local/remota=true.
7	Captura Wireshark	Parcial	Hay capturas DNS/TCP; falta correlación inequívoca con Render/JSON-RPC.
8	Especificación y parámetros	Documentado	Incluido en este reporte y README.
9	Análisis por capas	Parcial	Marco completo; evidencia final depende de captura filtrada.
10	Conclusiones	Documentado	Incluidas al final del reporte.

1. Arquitectura

La arquitectura separa la conversación, el protocolo y la lógica industrial. El LLM decide cuándo solicitar una herramienta; el anfitrión valida la solicitud, pide confirmación si existe un efecto sensible y delega en el cliente correspondiente.



Componente	Responsabilidad
GUI / terminal	Recibe configuración, mensajes y comandos de demostración.
Anfitrión	Mantiene historial de sesión, publica herramientas al LLM y coordina llamadas.
Cliente MCP	Implementa initialize, tools/list, tools/call, correlación por id y cierre.
Servidores MCP	Validan parámetros y ejecutan herramientas dentro de límites definidos.
Gemini	Genera respuestas y solicitudes estructuradas de herramientas.

2. Chatbot anfitrión, LLM y contexto

La clave se obtiene de LLM_API_KEY o del campo temporal de la GUI y nunca se incrusta en el repositorio. La configuración probada utiliza el proveedor compatible con OpenAI de Gemini, el modelo gemini-3.1-flash-lite y la URL base oficial de Google. El historial se conserva únicamente durante la ejecución activa y se descarta al cerrar el programa.

- Pregunta general probada: «¿Quién fue Alan Turing?».
- Continuación contextual probada: «¿En qué año nació?».
- Comandos de terminal: /help, /tools, /log y /exit.
- La GUI ofrece botones equivalentes para conversación, herramientas, log y demostraciones.

3. Implementación manual de MCP y JSON-RPC

El cliente construye y analiza mensajes JSON-RPC 2.0 directamente. Cada solicitud incluye un id de correlación; las notificaciones no esperan respuesta. stdout queda reservado para el transporte stdio y

los mensajes humanos se dirigen al sistema de logging. La versión negociada por el proyecto es 2025-11-25.

3.1 Secuencia de sesión

1. El cliente envía initialize con versión, capacidades e información del cliente.
2. El servidor responde con su versión y capacidades.
3. El cliente envía notifications/initialized sin id.
4. El cliente solicita tools/list y registra los esquemas disponibles.
5. Cuando el LLM selecciona una herramienta, el cliente envía tools/call con name y arguments.
6. El servidor devuelve result o un error JSON-RPC; el anfitrión entrega el resultado al LLM.

3.2 Ejemplos sanitizados

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "initialize",
  "params": {
    "protocolVersion": "2025-11-25",
    "capabilities": {},
    "clientInfo": {
      "name": "mcp-host",
      "version": "1.0"
    }
  }
}

{
  "jsonrpc": "2.0",
  "id": 4,
  "method": "tools/call",
  "params": {
    "name": "get_energy_report",
    "arguments": {
      "area": "Utilities"
    }
  }
}
```

4. Servidores MCP oficiales

4.1 Filesystem

El servidor Filesystem se ejecuta como subprocesso y queda limitado a demo_workspace, una ruta segura dentro del proyecto. Se emplea para crear y leer archivos de demostración sin exponer el resto del equipo.

4.2 Git

El servidor Git opera sobre un repositorio temporal creado específicamente para la demo. La prueba reproducible crea README.md, lo agrega al índice y genera un commit, sin modificar el repositorio principal.

```
python -m demos.filesystem_git_demo
```

Evidencia observada: el comando devolvió isError=false para git_add y git_commit, junto con el hash 3a47376019749da3cb00af9976eed2a48e740387 en el repositorio aislado de esa ejecución.

5. Servidor propio: Plant Energy MCP

El caso de uso industrial es el seguimiento del consumo energético de equipos de planta. La lógica utiliza datos deterministas en memoria para que las demostraciones local y remota sean reproducibles.

Herramienta	Parámetros principales	Resultado
list_equipment	area (opcional: Forming o Utilities)	Lista equipos y metadatos.

Herramienta	Parámetros principales	Resultado
record_energy_reading	equipment_id, timestamp, energy_kwh	Registra una lectura validada.
calculate_consumption	equipment_id, start_timestamp, end_timestamp	Calcula consumo en el intervalo.
detect_usage_alerts	equipo/área e intervalo según esquema	Detecta consumos sobre el umbral.
get_energy_report	area (opcional)	Resume equipos, consumo y estado.

5.1 Validación y errores

- Los identificadores deben corresponder a equipos conocidos.
- timestamp usa formato date-time y energy_kwh acepta valores entre 0 y 1 000 000 000.
- Las áreas válidas en los datos de demostración son Forming y Utilities.
- Los errores internos se convierten en respuestas seguras sin exponer trazas ni secretos.

6. Transportes local y remoto

Aspecto	Local	Remoto
Transporte	stdio	Streamable HTTP sobre HTTPS
Inicio	Subproceso Python	Servicio contenedorizado en Render
Endpoint MCP	stdin/stdout	POST /mcp
Salud	Estado del proceso	GET /health
Autenticación	Límite del proceso local	Authorization: Bearer [token]
Sesión	Vida del subproceso	Encabezado MCP-Session-Id
Lógica	PlantEnergyService	La misma PlantEnergyService

6.1 Endpoints remotos

- GET /health: devuelve {"status": "ok"} y no se confunde con MCP.

- POST /mcp: recibe JSON-RPC, valida Content-Type y Accept, y aplica límites de tamaño/timeout.
- GET /mcp: responde 405; DELETE /mcp permite cerrar una sesión cuando corresponde.
- URL comprobada: <https://plant-energy-mcp.onrender.com/mcp>.

6.2 Prueba de paridad

La demostración comparó `get_energy_report` en local y remoto. El resultado fue `equal=true`: tres equipos, 548.0 kWh de consumo total y `compressor-01` con estado `above_threshold`. Esta igualdad demuestra que los adaptadores de transporte reutilizan la misma lógica y conservan esquemas y comportamiento.

```
python -m demos.local_remote_parity_demo
```

7. Logging y controles de seguridad

El log MCP registra fecha y hora, servidor, transporte, dirección, método, id de correlación, duración y un resumen del resultado o error. Los campos sensibles se redactan. Puede consultarse con `/log` o con el botón «Ver log» de la GUI.

```
2026-08-27T17:21:18-06:00 | plant-remote | streamable-http | request | tools/call | id=4 | [token redactado]
```

- Filesystem restringido a una ruta explícita de demostración.
- Git restringido a un repositorio temporal en la demo automatizada.
- Confirmación previa para escrituras, commits y acciones sensibles solicitadas por el LLM.
- Timeouts, cierre limpio y aislamiento de fallos por servidor.

8. Pruebas y verificaciones

Verificación	Resultado real
<code>python -m unittest discover -v</code>	22 pruebas aprobadas en la verificación final del código.
<code>python -m compileall -q plant_energy_mcp mcp_host demos</code>	Compilación sintáctica sin errores.
<code>ruff check / format --check / mypy</code>	Verificaciones ejecutadas durante la implementación.
Chat Gemini	Pregunta general y continuación contextual respondidas.
Filesystem + Git	README creado, agregado y confirmado en repositorio aislado.
Render /health	Cinco respuestas consecutivas HTTP 200 observadas.

Verificación	Resultado real
Paridad local/remota	equal=true con salida determinista equivalente.

9. Captura y análisis con Wireshark

Las siguientes figuras corresponden a capturas reales suministradas durante la prueba. Se incluyen con una interpretación conservadora: solo se afirma lo que es visible en cada imagen.

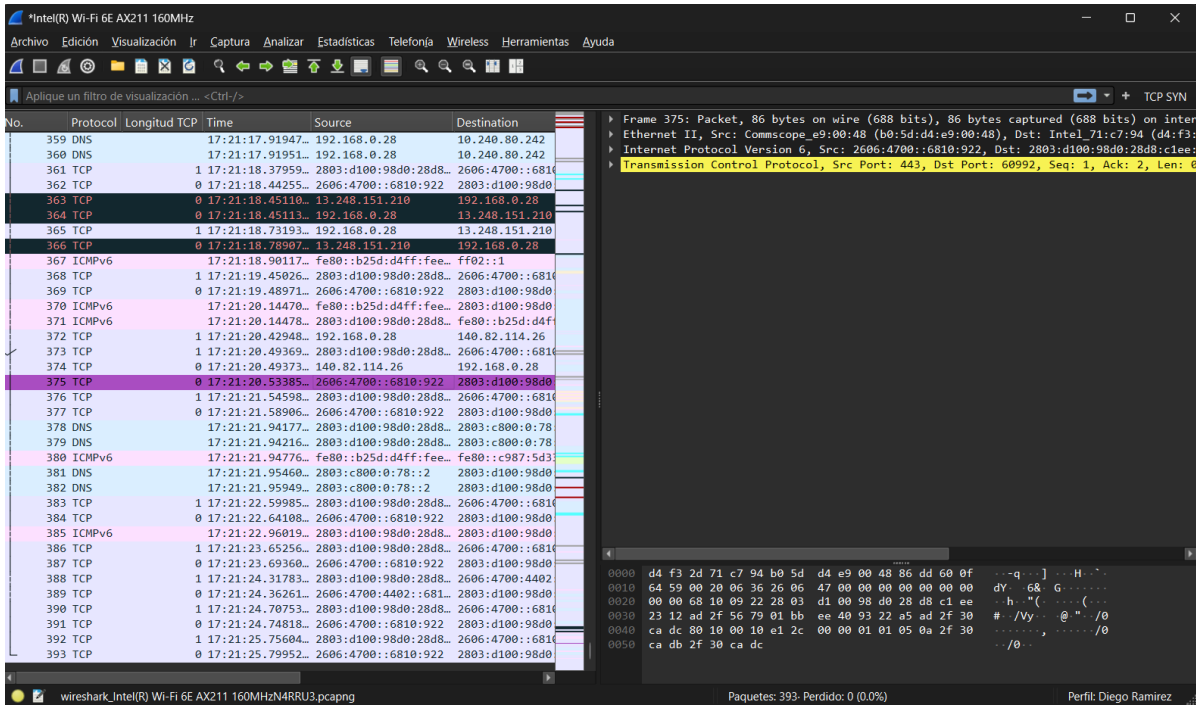


Figura 1. Captura general en la interfaz Intel(R) Wi-Fi 6E AX211 160MHz. Se observa tráfico DNS, TCP, IPv6 e ICMPv6 y un paquete TCP desde el puerto 443; la imagen no aplica un filtro por hostname, por lo que no permite atribuir por sí sola el flujo seleccionado a Render o Gemini.

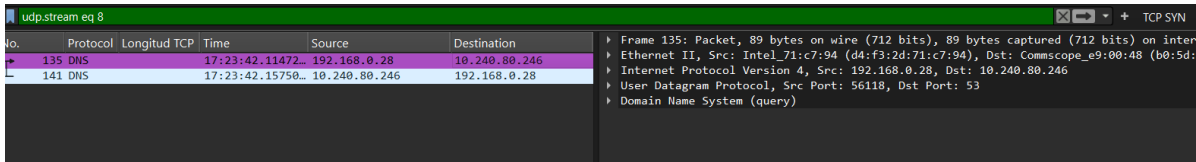


Figura 2. Seguimiento del flujo UDP 8. Se observa una consulta DNS desde 192.168.0.28:56118 hacia 10.240.80.246:53 y su respuesta. El nombre consultado no está expandido en la captura; por ello todavía no prueba que la consulta corresponda a plant-energy-mcp.onrender.com.

9.1 Datos observados

Elemento	Valor visible	Alcance de la evidencia
Interfaz	Intel(R) Wi-Fi 6E AX211 160MHz	Enlace de captura utilizado.

Elemento	Valor visible	Alcance de la evidencia
Host local IPv4	192.168.0.28	Origen de la consulta DNS.
Servidor DNS	10.240.80.246	Destino UDP 53 y origen de la respuesta.
Flujo DNS	udp.stream eq 8; paquetes 135 y 141	Consulta y respuesta visibles.
Puertos DNS	56118 → 53	Puerto efímero del cliente hacia DNS.
HTTPS/TCP	Paquete con puerto origen 443 y destino 60992	Tráfico cifrado visible, sin atribución de hostname.
Contenido JSON-RPC	No visible en estas imágenes	HTTPS protege la carga útil; requiere descifrado legítimo o correlación con log.

9.2 Explicación por capas

Capa de enlace

Wireshark captura tramas en la interfaz Wi-Fi. En el detalle de la figura 1 aparece Ethernet II, la representación que usa Wireshark para mostrar direcciones de enlace en esta captura. Estas direcciones identifican el salto local, no al servidor de Render extremo a extremo.

Capa de red

Se observan paquetes IPv4 e IPv6. IPv4 transporta la consulta DNS entre 192.168.0.28 y 10.240.80.246; la captura general también muestra conexiones IPv6. IP permite el encaminamiento entre redes, pero la imagen actual no muestra la resolución de Render expandida para ligar una dirección al servicio.

Capa de transporte

DNS usa UDP puerto 53 en la figura 2. La comunicación HTTPS usa TCP puerto 443; TCP proporciona establecimiento de conexión, orden, retransmisión y entrega confiable antes de TLS. Los puertos efímeros identifican el extremo cliente de cada conversación.

Capa de aplicación

DNS resuelve nombres y HTTPS transporta HTTP protegido por TLS. Dentro de POST /mcp viajan mensajes JSON-RPC como initialize, tools/list y tools/call. Debido al cifrado, esos métodos no se leen en las capturas actuales; sí pueden demostrarse mediante el log MCP con marcas de tiempo coincidentes o, si el cliente lo soporta, mediante SSLKEYLOGFILE utilizado legítimamente y una captura sanitizada.

9.3 Procedimiento reproducible para completar la evidencia

1. Cerrar aplicaciones que generen tráfico innecesario y abrir Wireshark como administrador.
2. Seleccionar Intel(R) Wi-Fi 6E AX211 160MHz e iniciar una captura nueva.
3. Aplicar el filtro `dns.qry.name contains "plant-energy-mcp.onrender.com"` y pulsar «Comprobar Render» en la GUI.
4. Expandir DNS en la consulta y respuesta; anotar la IP resuelta sin inventarla.
5. Filtrar `ip.addr == IP_OBSERVADA && tcp.port == 443`; ejecutar «Reporte remoto» una sola vez.
6. Capturar el handshake y, en paralelo, abrir «Ver log» para correlacionar las operaciones JSON-RPC.
7. Guardar una copia sanitizada .pcapng y las capturas finales; no incluir claves ni tokens.

10. Dificultades, soluciones y lecciones aprendidas

Dificultad real	Solución aplicada / lección
Errores HTTP 400 y 503 de Gemini	Se corrigió proveedor/clave/modelo y se usó gemini-3.1-flash-lite; 503 puede ser temporal y requiere reintentos claros.
Cloud Run exigía facturación	Se preparó un contenedor reproducible y se desplegó en Render sin confundir la API de Gemini con un servicio de nube.
404 inicial en Render	Se verificó /health y la ruta exacta /mcp antes de repetir la paridad.
Complejidad de comandos	Se agregó una GUI de demostración que centraliza configuración, chat, herramientas, log y pruebas.
HTTPS oculta JSON-RPC	Se distingue evidencia de transporte de contenido de aplicación y se propone correlación temporal o descifrado legítimo.

11. Conclusiones

El proyecto demuestra que MCP puede implementarse manualmente sobre JSON-RPC sin depender de un SDK. La separación entre anfitrión, clientes, transportes y lógica industrial permitió ejecutar el mismo servidor en local y remoto con resultados equivalentes. El contexto por sesión y la integración de herramientas convierten al LLM en un coordinador, mientras que las validaciones, límites de ruta, confirmaciones y redacción de secretos reducen riesgos.

La principal evidencia pendiente es la captura Wireshark filtrada y correlacionada con Render. Las capturas actuales sirven como evidencia parcial de DNS y HTTPS, pero no deben presentarse como lectura directa de JSON-RPC. Completar esa correlación permitirá cerrar los puntos 7 y 9 con evidencia verificable.

12. Fuentes y atribuciones

- Model Context Protocol. Specification 2025-11-25.
<https://modelcontextprotocol.io/specification/2025-11-25>
- JSON-RPC Working Group. JSON-RPC 2.0 Specification. <https://www.jsonrpc.org/specification>
- Google AI for Developers. Gemini API documentation. <https://ai.google.dev/gemini-api/docs>
- Wireshark Foundation. Wireshark User's Guide.
https://www.wireshark.org/docs/wsug_html_chunked/
- Render. Web Services documentation. <https://render.com/docs/web-services>
- Model Context Protocol servers repository. <https://github.com/modelcontextprotocol/servers>